

Джек Рашер в докладе «Stop Writing Dead Programs» диагностирует фундаментальную проблему: современное программирование, несмотря на технологический прогресс, сохранило логику эпохи перфокарт. Цикл «написал — скомпилировал — запустил — получил ошибку — перезапустил» превращает программу в «мёртвый артефакт», оторванный от разработчика в момент выполнения. Рашер противопоставляет этому «живое» интерактивное программирование, где среда и программа неразделимы: можно изменять код в работающей системе, исследовать состояние памяти и визуализировать данные в реальном времени. В качестве образцов приводятся Smalltalk, Common Lisp, Erlang, Clojure, а также современные инструменты вроде Clerk и Hazel с typed holes. Отдельно критикуется культ статической типизации: автор утверждает, что отлаживаемость важнее доказанной корректности, а спецификация всегда неполна.

Идеи Рашера представляются мне чрезвычайно актуальными и в значительной мере достоверными. Как студент, работающий с C++ и ассемблером, я ежедневно сталкиваюсь с разорванным циклом обратной связи: часы отладки ради исправления одной строки. Аргумент про визуальную кору особенно убедителен — пример Datasaurus наглядно показывает, как параллельное восприятие превосходит последовательное чтение чисел. Однако некоторые утверждения требуют нюансов: индустриальный опыт TypeScript и Rust демонстрирует реальную пользу статических типов в больших проектах, а критика Go за «отсутствие интроспекции» игнорирует мощные инструменты вроде pprof и delve. Тем не менее, даже если не принимать все тезисы буквально, доклад выполняет важную функцию — заставляет пересмотреть привычные ограничения как данность.

На мой взгляд, главная ценность доклада — не конкретные рекомендации, а приём острания: показать, насколько наши инструменты определены инерцией, а не задачами. Индустрия оказалась в ловушке собственных стандартов: программисты гордятся способностью «моделировать процессор в уме», хотя технологии позволяют иначе. Вместе с тем, я не разделяю надежду, что «живое программирование» полностью вытеснит batch-подход. Современная командная разработка требует воспроизводимости и детерминизма (Docker, CI/CD), которые плохо сочетаются с REPL для пятидесяти разработчиков. Будущее, скорее всего, гибридно: интерактивность ноутбуков и structural editors — для исследования и обучения, строгость и изоляция — для продакшена. Доклад полезен именно тем, что напоминает: прогресс — это не только новые библиотеки, но и переосмысление нормы. Как молодое поколение, мы можем выбирать инструменты осознанно, не принимая «костыли» за неизбежность.